
Project Anex (Group 11):

Object Detection and Classification System Using

ESP32-CAM

Date: 20/12/2024

Course: Object-Oriented Programming (OOP)

Group Members:

Nguyen Do Nguyen - 10223056

Nguyen Quoc An - 10223002

Nguyen Dinh Thai Tue - 10223071

Nguyen Xuan Son - 10222042

Table of content:

1. ESP-Cam Code

2. OOP Program

2.1. Communicator.cpp

2.2. Communicator.h

2.3. Manager.cpp

2.4. Manager.h

2.5. Main.cpp

1. ESP-Cam code

```
#include <tu3t-project-1_inferencing.h>
#include "edge-impulse-sdk/dsp/image/image.hpp"

#include "esp_camera.h"

// Select camera model - find more camera models in camera_pins.h file here
// https://github.com/espressif/arduino-esp32/blob/master/libraries/ESP32/examples/Camera/CameraWebServer/camera_pins.h

// #define CAMERA_MODEL_ESP_EYE // Has PSRAM
#define CAMERA_MODEL_AI_THINKER // Has PSRAM

#if defined(CAMERA_MODEL_ESP_EYE)
#define PWDN_GPIO_NUM    -1
#define RESET_GPIO_NUM   -1
#define XCLK_GPIO_NUM     4
#define SIOD_GPIO_NUM     18
#define SIOC_GPIO_NUM     23

#define Y9_GPIO_NUM       36
#define Y8_GPIO_NUM       37
#define Y7_GPIO_NUM       38
#define Y6_GPIO_NUM       39
#define Y5_GPIO_NUM       35
#define Y4_GPIO_NUM       14
#define Y3_GPIO_NUM       13
#define Y2_GPIO_NUM       34
#define VSYNC_GPIO_NUM    5
#define HREF_GPIO_NUM     27
#define PCLK_GPIO_NUM     25

#elif defined(CAMERA_MODEL_AI_THINKER)
#define PWDN_GPIO_NUM     32
#define RESET_GPIO_NUM    -1
#define XCLK_GPIO_NUM      0
#define SIOD_GPIO_NUM     26
#define SIOC_GPIO_NUM     27

#define Y9_GPIO_NUM       35
#define Y8_GPIO_NUM       34
#define Y7_GPIO_NUM       39
#define Y6_GPIO_NUM       36
#define Y5_GPIO_NUM       21
#define Y4_GPIO_NUM       19
#define Y3_GPIO_NUM       18
#define Y2_GPIO_NUM       5
#define VSYNC_GPIO_NUM    25
```

```

#define HREF_GPIO_NUM      23
#define PCLK_GPIO_NUM      22

#else
#error "Camera model not selected"
#endif

/* Constant defines ----- */
#define EI_CAMERA_RAW_FRAME_BUFFER_COLS      320
#define EI_CAMERA_RAW_FRAME_BUFFER_ROWS      240
#define EI_CAMERA_FRAME_BYTE_SIZE            3

/* Private variables ----- */
static bool debug_nn = false; // Set this to true to see e.g. features generated from the raw signal
static bool is_initialised = false;
uint8_t *snapshot_buf; //points to the output of the capture

static camera_config_t camera_config = {
    .pin_pwdn = PWDN_GPIO_NUM,
    .pin_reset = RESET_GPIO_NUM,
    .pin_xclk = XCLK_GPIO_NUM,
    .pin_sscb_sda = SIOD_GPIO_NUM,
    .pin_sscb_scl = SIOC_GPIO_NUM,

    .pin_d7 = Y9_GPIO_NUM,
    .pin_d6 = Y8_GPIO_NUM,
    .pin_d5 = Y7_GPIO_NUM,
    .pin_d4 = Y6_GPIO_NUM,
    .pin_d3 = Y5_GPIO_NUM,
    .pin_d2 = Y4_GPIO_NUM,
    .pin_d1 = Y3_GPIO_NUM,
    .pin_d0 = Y2_GPIO_NUM,
    .pin_vsync = VSYNC_GPIO_NUM,
    .pin_href = HREF_GPIO_NUM,
    .pin_pclk = PCLK_GPIO_NUM,

    //XCLK 20MHz or 10MHz for OV2640 double FPS (Experimental)
    .xclk_freq_hz = 20000000,
    .ledc_timer = LEDC_TIMER_0,
    .ledc_channel = LEDC_CHANNEL_0,

    .pixel_format = PIXFORMAT_JPEG, //YUV422,GRAYSCALE,RGB565,JPEG
    .frame_size = FRAMESIZE_QVGA,    //QQVGA-UXGA Do not use sizes above QVGA when not JPEG

    .jpeg_quality = 12, //0-63 lower number means higher quality
    .fb_count = 1,      //if more than one, i2s runs in continuous mode. Use only with JPEG
    .fb_location = CAMERA_FB_IN_PSRAM,
    .grab_mode = CAMERA_GRAB_WHEN_EMPTY,
};

```

```

/* Function definitions ----- */
bool ei_camera_init(void);
void ei_camera_deinit(void);
bool ei_camera_capture(uint32_t img_width, uint32_t img_height, uint8_t *out_buf) ;

/**
 * @brief      Arduino setup function
 */
void setup()
{
    // put your setup code here, to run once:
    Serial.begin(115200);

    //comment out the below line to start inference immediately after upload
    while (!Serial);
    Serial.println("Edge Impulse Inferencing Demo");
    if (ei_camera_init() == false) {
        ei_printf("Failed to initialize Camera!\r\n");
    }
    else {
        ei_printf("Camera initialized\r\n");
    }

    ei_printf("\nStarting continious inference in 2 seconds...\n");
    ei_sleep(2000);
}

/**
 * @brief      Get data and run inferencing
 *
 * @param[in]  debug  Get debug info if true
 */
void loop()
{
    // instead of wait_ms, we'll wait on the signal, this allows threads to cancel us...
    if (ei_sleep(5) != EI_IMPULSE_OK) {
        return;
    }

    snapshot_buf = (uint8_t*)malloc(EI_CAMERA_RAW_FRAME_BUFFER_COLS * EI_CAMERA_RAW_FRAME_BUFFER_ROWS * EI_CAMERA_FRAME_BYTE_SIZE);

    // check if allocation was successful
    if(snapshot_buf == nullptr) {
        return;
    }

    ei::signal_t signal;
    signal.total_length = EI_CLASSIFIER_INPUT_WIDTH * EI_CLASSIFIER_INPUT_HEIGHT;
    signal.get_data = &ei_camera_get_data;

```

```

if (ei_camera_capture((size_t)EI_CLASSIFIER_INPUT_WIDTH, (size_t)EI_CLASSIFIER_INPUT_HEIGHT, snapshot_buf) == false) {
    ei_printf("Failed to capture image\r\n");
    free(snapshot_buf);
    return;
}

// Run the classifier
ei_impulse_result_t result = { 0 };

EI_IMPULSE_ERROR err = run_classifier(&signal, &result, debug_nn);
if (err != EI_IMPULSE_OK) {
    ei_printf("ERR: Failed to run classifier (%d)\n", err);
    return;
}

#if EI_CLASSIFIER_OBJECT_DETECTION == 1
    if (result.bounding_boxes_count > 0){
        float finVal = -1.0;
        char finLab[100];
        // ei_printf("Object detection bounding boxes:\r\n");
        for (uint32_t i = 0; i < result.bounding_boxes_count; i++) {

            ei_impulse_result_bounding_box_t bb = result.bounding_boxes[i];
            if (bb.value == 0) {
                continue;
            }
            if (bb.value > finVal){
                finVal = bb.value;
                strcpy(finLab, bb.label);
            }
        }
        ei_printf("%s\n", finLab);
    }

    // Print the prediction results (classification)
#else
    ei_printf("Predictions:\r\n");
    for (uint16_t i = 0; i < EI_CLASSIFIER_LABEL_COUNT; i++) {
        ei_printf("  %s: ", ei_classifier_inferencing_categories[i]);
        ei_printf("%.5f\r\n", result.classification[i].value);
    }
#endif

    // Print anomaly result (if it exists)
#if EI_CLASSIFIER_HAS_ANOMALY
    ei_printf("Anomaly prediction: %.3f\r\n", result.anomaly);
#endif

```

```

#if EI_CLASSIFIER_HAS_VISUAL_ANOMALY
    ei_printf("Visual anomalies:\r\n");
    for (uint32_t i = 0; i < result.visual_ad_count; i++) {
        ei_impulse_result_bounding_box_t bb = result.visual_ad_grid_cells[i];
        if (bb.value == 0) {
            continue;
        }
        ei_printf("  %s (%f) [ x: %u, y: %u, width: %u, height: %u ]\r\n",
            bb.label,
            bb.value,
            bb.x,
            bb.y,
            bb.width,
            bb.height);
    }
#endif

    free(snapshot_buf);
}

/**
 * @brief   Setup image sensor & start streaming
 *
 * @retval  false if initialisation failed
 */
bool ei_camera_init(void) {

    if (is_initialised) return true;

#if defined(CAMERA_MODEL_ESP_EYE)
    pinMode(13, INPUT_PULLUP);
    pinMode(14, INPUT_PULLUP);
#endif

    //initialize the camera
    esp_err_t err = esp_camera_init(&camera_config);
    if (err != ESP_OK) {
        Serial.printf("Camera init failed with error 0x%x\n", err);
        return false;
    }

    sensor_t * s = esp_camera_sensor_get();
    // initial sensors are flipped vertically and colors are a bit saturated
    if (s->id.PID == OV3660_PID) {
        s->set_vflip(s, 1); // flip it back
        s->set_brightness(s, 1); // up the brightness just a bit
        s->set_saturation(s, 0); // lower the saturation
    }
}

```

```

#if defined(CAMERA_MODEL_MSSTACK_WIDE)
    s->set_vflip(s, 1);
    s->set_hmirror(s, 1);
#elif defined(CAMERA_MODEL_ESP_EYE)
    s->set_vflip(s, 1);
    s->set_hmirror(s, 1);
    s->set_awb_gain(s, 1);
#endif

    is_initialised = true;
    return true;
}

/**
 * @brief      Stop streaming of sensor data
 */
void ei_camera_deinit(void) {

    //deinitialize the camera
    esp_err_t err = esp_camera_deinit();

    if (err != ESP_OK)
    {
        ei_printf("Camera deinit failed\n");
        return;
    }

    is_initialised = false;
    return;
}

/**
 * @brief      Capture, rescale and crop image
 *
 * @param[in]  img_width    width of output image
 * @param[in]  img_height   height of output image
 * @param[in]  out_buf      pointer to store output image, NULL may be used
 *                          if ei_camera_frame_buffer is to be used for capture and resize/cropping.
 *
 * @retval     false if not initialised, image captured, rescaled or cropped failed
 */
bool ei_camera_capture(uint32_t img_width, uint32_t img_height, uint8_t *out_buf) {
    bool do_resize = false;

    if (!is_initialised) {
        ei_printf("ERR: Camera is not initialized\r\n");
        return false;
    }

```

```

}

camera_fb_t *fb = esp_camera_fb_get();

if (!fb) {
    ei_printf("Camera capture failed\n");
    return false;
}

bool converted = fmt2rgb888(fb->buf, fb->len, PIXFORMAT_JPEG, snapshot_buf);

esp_camera_fb_return(fb);

if(!converted){
    ei_printf("Conversion failed\n");
    return false;
}

if ((img_width != EI_CAMERA_RAW_FRAME_BUFFER_COLS)
    || (img_height != EI_CAMERA_RAW_FRAME_BUFFER_ROWS)) {
    do_resize = true;
}

if (do_resize) {
    ei::image::processing::crop_and_interpolate_rgb888(
        out_buf,
        EI_CAMERA_RAW_FRAME_BUFFER_COLS,
        EI_CAMERA_RAW_FRAME_BUFFER_ROWS,
        out_buf,
        img_width,
        img_height);
}

return true;
}

static int ei_camera_get_data(size_t offset, size_t length, float *out_ptr)
{
    // we already have a RGB888 buffer, so recalculate offset into pixel index
    size_t pixel_ix = offset * 3;
    size_t pixels_left = length;
    size_t out_ptr_ix = 0;

    while (pixels_left != 0) {
        // Swap BGR to RGB here
        // due to https://github.com/espressif/esp32-camera/issues/379
        out_ptr[out_ptr_ix] = (snapshot_buf[pixel_ix + 2] << 16) + (snapshot_buf[pixel_ix + 1] << 8) + snapshot_buf[pixel_ix];

        // go to the next pixel
    }
}

```

```

        out_ptr_ix++;
        pixel_ix+=3;
        pixels_left--;
    }
    // and done!
    return 0;
}

#ifdef EI_CLASSIFIER_SENSOR || EI_CLASSIFIER_SENSOR != EI_CLASSIFIER_SENSOR_CAMERA
#error "Invalid model for current sensor"
#endif

```

2. OOP Program

2.1 Communicator.cpp

```

#include <iostream>
#include <fstream>
#include <fcntl.h>
#include <termios.h>
#include <unistd.h>
#include <cstring>
#include "communicator.h"

void configurePort(int fd) {
    struct termios tty;

    // Get the current configuration of the serial port
    if (tcgetattr(fd, &tty) != 0) {
        perror("Error from tcgetattr");
        exit(EXIT_FAILURE);
    }

    // Set the baud rate for both input and output
    cfsetospeed(&tty, B115200);
    cfsetispeed(&tty, B115200);

    // Configure the serial port
    tty.c_cflag = (tty.c_cflag & ~CSIZE) | CS8; // 8 data bits
    tty.c_iflag &= ~IGNBRK; // Ignore break condition
    tty.c_lflag = 0; // No local modes
    tty.c_oflag = 0; // No output processing
    tty.c_cc[VMIN] = 1; // Read at least 1 character
    tty.c_cc[VTIME] = 0; // No timeout
}

```

```

tty.c_cflag |= (CLOCAL | CREAD);    // Enable receiver and set local mode
tty.c_cflag &= ~(PARENB | PARODD); // Disable parity
tty.c_cflag &= ~CSTOPB;             // 1 stop bit
tty.c_cflag &= ~CRTSCTS;            // No hardware flow control

// Apply the configuration
if (tcsetattr(fd, TCSANOW, &tty) != 0) {
    perror("Error from tcsetattr");
    exit(EXIT_FAILURE);
}
}

//The function the main program uses to retrieve data from the camera
std::string getComp() {
    const char* portName = "/dev/ttyACM0";
    char buffer[255];
    int fd;

    // Open the serial port
    fd = open(portName, O_RDWR | O_NOCTTY | O_SYNC);
    if (fd < 0) {
        return "Error opening port";
    }

    // Configure the port
    configurePort(fd);

    // Clear any data in the input buffer
    if (tcflush(fd, TCIFLUSH) != 0) {
        close(fd);
        return "Error flushing input buffer";
    }

    std::string res = "";
    do {
        memset(buffer, 0, sizeof(buffer)); // Clear the buffer

        // Read data from the serial port
        int bytesRead = read(fd, buffer, sizeof(buffer) - 1); // Fixed `sizeof(0)` to `sizeof(buffer)`
        if (bytesRead > 0) {
            buffer[bytesRead] = '\0'; // Null-terminate the string
            for (int i = 0; i < bytesRead; i++) {
                if (buffer[i] == '\n') {
                    close(fd); // Close the serial port
                    return res;
                }
                else res += buffer[i];
            }
        } else if (bytesRead < 0) {
            std::cout << "Error reading from serial port";

```

```

        break;
    }
} while (true);
std::cout << "closed file";
close(fd); // Close the serial port
return "zero";
}

```

2.2 Communicator.h

```

#ifndef COMMUNICATOR_H
#define COMMUNICATOR_H
#include <string>

std::string getComp();

#endif //COMMUNICATOR_H

```

2.3 Manager.cpp

```

#include<iostream>
#include<string>
#include<cstdio>
#include<fstream>
#include<vector>
#include<sstream>
#include "Manager.h"

using namespace std;

//Constructors
Product::Product() {
    //Assign default values
    name = "-";
    price = -1;
    desc = "";
    StoreX = -1;
    StoreY = -1;
}

Product::Product(const string &str) {
    //Get description
    ifstream file("Desc.txt"); // Open the file
    if (!file.is_open()) {      // Check if the file opened successfully
        std::cerr << "Error: Could not open the file." << std::endl;
    }

    string descScan;
    while (getline(file, descScan)){

```

```

        if(str == descScan) {
            getline(file, desc);
            break;
        }
        else getline(file, descScan);
    }
    file.close();

    name = str;

    // Get Price
    ifstream fileP("Price.txt"); // Open the file
    if (!fileP.is_open()) {        // Check if the file opened successfully
        std::cerr << "Error: Could not open the file." << std::endl;
    }

    price = -1;
    int temp;
    string val, num;
    while (getline(fileP, val)) {
        if(val == str) {
            fileP >> temp;
            price = temp;
            break;
        }
    }
    fileP.close();

    //Get Location
    ifstream filei("Location.txt"); // Open the file
    string item;
    while (filei >> item) {
        int cordX, cordY;
        fileP >> cordX >> cordY;
        if (item == str) {
            StoreX = cordX;
            StoreY = cordY;
        }
    }
    filei.close();
}

string Product::getName() const {
    return name;
}

string Product::getDesc() const{
    return desc;
}

int Product::getPrice() const{
    return price;
}

```

```

}

void Product::editPrice(const int &newP){
    ifstream filein("Price.txt"); // Open the file
    if (!filein.is_open()) {        // Check if the file opened successfully
        std::cerr << "Error: Could not open the file." << std::endl;
    }

    int cnt = 1, pos = -1;
    string item[100],temp;
    int val[100];
    //Scan Price.txt file, assigning values into two arrays
    do{
        if (filein.eof()) break;
        getline(filein, temp);
        if (cnt %2 != 0) {
            item[cnt/2] = temp;
            if (item[cnt/2] == name) pos = cnt/2;
        }
        else {
            val[(cnt/2)-1] = stoi(temp);
        }

    }while (cnt +=1);

    filein.close();

    if (pos > 0) {
        val[pos] = newP;
        price = newP;
    }

    //Rewrite content to the data file
    ofstream outfile("Price.txt");
    for (int i = 0; i < cnt/2; i++) {
        if (i != 0) outfile << "\n";
        outfile << item[i] << "\n" << val[i];
    }
    outfile.close();
}

void Product::editDesc(const string &newDesc) {
    ifstream filein("Desc.txt"); // Open the file
    if (!filein.is_open()) {        // Check if the file opened successfully
        std::cerr << "Error: Could not open the file." << std::endl;
    }

    int cnt = 1, pos = -1;
    string item[100],temp;
    string val[100];

```

```

//Scan Price.txt file, assigning values into two arrays
do{
    if (filein.eof()) break;
    getline(filein, temp);
    if (cnt %2 != 0) {
        item[cnt/2] = temp;
        if (item[cnt/2] == name) pos = cnt/2;
    }
    else {
        val[(cnt/2)-1] = temp;
    }

}while (cnt +=1);

filein.close();

if (pos > 0) {
    val[pos] = newDesc;
    desc = newDesc;
}

//Rewrite content to the data file
ofstream outfile("Desc.txt"); // Open the file
for (int i = 0; i < cnt/2; i++) {
    if (i != 0) outfile << "\n";
    outfile << item[i] << "\n" << val[i];
}
outfile.close();
}

//Operators
Product& Product::operator=(const string& str) {
    Product temp(str);
    this->name = temp.name;
    this->desc = temp.desc;
    this->price = temp.price;
    return *this;
}

```

2.4 Manager.h

```
#ifndef MANAGER_H
#define MANAGER_H

#include<iostream>
#include<string>
#include<cstdio>
#include<fstream>
#include<vector>
#include<sstream>

using namespace std;

//define class
class Product {
private:
    string desc, name;
    int price;
    int StoreX, StoreY;

public:
    //Constructor
    Product();
    Product(const string &str);

    //Functions
    string getName() const;
    string getDesc() const;
    int getPrice() const;
    void editPrice(const int &newP);
    void editDesc(const string &newDesc);
    void isValidLocation();

    //Operators
    Product& operator=(const string& str);
};

#endif //MANAGER_H
```

2.5 Main.cpp

```
#include "communicator.h"
#include "Manager.h"
#include <iostream>
#include <vector>
#include <fstream>

using namespace std;

int main() {
    while (true) {
        int option;

        cout << "\n\n\n\n\nSelect an operation mode: \n1 - Component identifier\n2 - Component edit\n3 - Checkout mode\n4 - Storage manager\n5 - Exit\n\n-> ";

        cin >> option;
        if (option == 1) {
            cout << "Put the component under the camera...\n";
            cout << "Scanning for item...\n";

            string tag = getComp();
            Product prd(tag);

            cout << "\n-----\nPredicted item: " << prd.getName() << "\n";
            cout << "Quick description: " << prd.getDesc() << "\nPrice: " << prd.getPrice() << "\n";

            cin.ignore(); cin.get();
        }
        else if (option == 2) {
            string tag;
            while (true) {
                cout << "Put the component under the camera...\n";
                cout << "Scanning for item...\n";

                tag = getComp();

                cout << "\nPredicted item: " << tag << "\nEnter '1' to change description, '2' to change price, '3' to rescan:\n-> ";
                int choice; cin.ignore(); cin >> choice;

                if (choice == 3) continue;

                Product prd(tag);

                if (choice == 1) {
                    string newval;
                    cin.ignore();

                    cout << "Enter new description: "; getline(cin, newval);
                    prd.editDesc(newval);

                    cout << "The description of <" << prd.getName() << "> has been modified\n";
                }

                if (choice == 2) {
                    int newval;

                    cout << "Enter new price: "; cin >> newval;

                    prd.editPrice(newval);

                    cout << "The price of <" << prd.getName() << "> is now " << prd.getPrice() << "\n";
                }
            }
            break;
        }
    }
}
```

```

    }
    cin.ignore();cin.get();
}
else if (option == 3) {
    vector<Product> einkaufsliste;
    int total = 0;
    while (true) {
        cout << "Current Basket:\t<" << einkaufsliste.size() << "> item(s)\tTotal price: " << total << "\n";
        cout << "Pick an option:\n1 - Show basket\n2 - Add item to basket\n3 - Remove item\n4 - Checkout\n-> ";
        int mode; cin >> mode;
        if (mode == 1 || mode == 3 || mode ==4) {
            if (mode == 4) cout << "Final list:\n";
            cout << "No.\t\tName\t\tPrice\n";
            for (int i=0; i<einkaufsliste.size(); i++) {
                cout << i+1 << "\t\t" << einkaufsliste[i].getName() << "\t\t" << einkaufsliste[i].getPrice() << "\n";
            }
            if (mode == 3) {
                cout << "Enter the index of the item to be removed\n-> ";
                int index; cin >> index;
                total -= einkaufsliste[index].getPrice();
                einkaufsliste.erase(einkaufsliste.begin() + index);
                cout << "Item deleted!";
                cin.ignore(); cin.get();
                continue;
            }
            if (mode == 4 || mode ==1) {
                cout << "Total basket price: " << total << "\nPress any key to continue";
                cin.ignore(); cin.get();
                if (mode == 4) break;
            }
        }
    }
}
else if (mode == 2) {
    while (true) {
        cout << "Put the component under the camera...\n";
        cout << "Scanning for item...\n";
        string tag = getComp();
        cout << "\nPredicted item: " << tag << "\nEnter '1' recans, '2' to accept\n-> ";
        int choice; cin.ignore(); cin >> choice;
        if (choice == 2) {
            Product add(tag);
            einkaufsliste.push_back(add);
            total += add.getPrice();
            break;
        }
    }
}
}
}
}

```

```

else if (option == 4) {
    while (true) {
        int modeS, NumWH, NumSlot;
        Product nll;
        ifstream fileP("Location.txt");
        fileP >> NumWH >> NumSlot;
        // cout << NumWH << " " << NumSlot << "\n";
        vector<vector<Product>> Storage; Storage.resize(NumWH);
        for (int i=0; i<NumWH; i++) {
            Storage[i].resize(NumSlot);
            for (int j=0; j<NumSlot; j++) Storage[i][j] = nll;
        }
        string item;
        while (fileP >> item) {
            // cout << item << "---00000---";
            int cordX, cordY;
            fileP >> cordX >> cordY;
            Product tmp(item);
            Storage[cordX][cordY] = tmp;
        }
        fileP.close();
        cout << "Pick a mode: \n1 - Show map\n2 - Add/Move item\n-> "; cin >> modeS;
        if (modeS == 1) {
            cout << "\t\t\t";
            for (int j=0; j<NumSlot; j++) cout << j << "\t\t";
            cout << "\n";
            for (int i=0; i<NumWH; i++) {
                cout << "Storage " << char(65+i) << "\t";
                for (int j=0; j<NumSlot; j++) {
                    cout << Storage[i][j].getName().substr(0, 3) << "\t\t";
                }
                cout << "\n";
            }
            for (int i=0; i<NumWH; i++) {
                for (int j=0; j<NumSlot; j++) {
                    if (Storage[i][j].getName() != "-") {
                        cout << Storage[i][j].getName().substr(0, 3) << "\t\t" << Storage[i][j].getName() << "\n";
                    }
                }
            }
            cin.ignore(); cin.get();
        }
        else if (modeS == 2) {
            // cout << "";
            cout << "Enter component name: ";
            string tag;
            cin.ignore(); getline(cin, tag);
            while (true) {
                char SName;
                cout << "Enter the storage (A->Z): "; cin >> SName;
            }
        }
    }
}

```

```

        int posx = int(SName-'A');
        int posy;
        cout << "Enter the storage slot: "; cin >> posy;
        if (Storage[posx][posy].getName() == "-") {
            for (int i=0; i<NumWH; i++) {
                for (int j=0; j<NumSlot; j++) {
                    if (Storage[i][j].getName() == tag) Storage[i][j] = null;
                }
            }
            Product newsign(tag);
            Storage[posx][posy] = newsign;
            break;
        }
        else cout << "Invalid location! Try again...";
        cin.ignore(); cin.get();
    }
    ofstream file0("Location.txt");
    file0 << NumWH << " " << NumSlot << "\n";
    for (int i=0; i<NumWH; i++) {
        for (int j=0; j<NumSlot; j++) {
            if (Storage[i][j].getName() != "-") {
                file0 << Storage[i][j].getName() << "\n" << i << " " << j << "\n";
            }
        }
    }
    file0.close();
}
else {
    break;
}
}

}

else if (option == 5) break;
else cout << "\n\nInvalid option, please try again.\n";
}

return 0;
}

```